# Uživatelský manuál Marvin

## Kompletní průvodce lokální chatovací a vývojovou stanicí

| | |
|---|---|
| **Verze aplikace** | 1.20.0 |
| **Uživatelský manuál** | 2026 |

Local AI · Qwen 3.8 · Ornith 1.5 · Windows 11 · RTX 5090

# Obsah

1. Co je Marvin 3

2. Požadavky a instalace 3

3. Prohlídka desktopového rozhraní 5

4. Modely, KV cache, kontext a thinking 9

5. Odesílání, steering, Stop a průběh 11

6. Pracovní režimy 12

7. Projekty a workspace 12

8. Správa chatů 12

9. Kontext, komprese a pins 13

10. Tři vrstvy paměti 13

11. Volitelné skills 13

12. Internet a výzkum 14

13. Psaní a export dokumentů 15

14. Vývojový workflow 15

15. Ovládání počítače 17

16. Autonomie a potvrzování 18

17. Terminálové UI a CLI 18

18. Úložiště, zálohy a logy 19

19. Řešení problémů 20

20. Reference nástrojů 21

21. Praktické příklady 23

22. Provozní omezení 23

23. Rozšířené pracovní operace 24

# 1. Co je Marvin

Marvin je soukromá desktopová aplikace pro Windows, která provozuje lokální jazykové modely jako obecný chat, výzkumného asistenta, partnera pro psaní, coding agenta a operátora počítače. Inference modelu běží na pracovní stanici přes `llama.cpp`; konverzace a projektové soubory zůstávají na lokálním disku.

Aplikace je navržena pro jednoho uživatele a jeden aktivní model. Nepoužívá paralelní modelové agenty. Aktivní model ale může paralelně spouštět nezávislé čtecí nástroje a může ponechat dlouhé příkazy běžet na pozadí.

POZNÁMKA: Inference modelu je lokální. Když model použije `web_search` nebo `web_fetch`, aplikace provádí běžné internetové požadavky na vyhledávače a veřejné webové stránky. Pokud potřebujete zcela offline relaci, vypněte web v `config.yaml` nebo nepoužívejte výzkumné/webové požadavky.

## Hlavní schopnosti

- Běžná konverzace bez programátorského chování.
- Výzkum z více webových a dokumentových zdrojů se závěrečnou syntézou.
- Psaní a revize scénářů, článků, reportů, treatmentů a dalších dokumentů.
- Lokální vývoj projektů s nástroji pro soubory, Git, shell, testy, obrázky a rollback.
- Ovládání aplikací ve Windows podle screenshotů.
- Trvalé projekty, více chatů v projektu, třívrstvá paměť, připnuté soubory a volitelné skills.
- Anglické a české rozhraní, které lze přepnout bez ztráty aktivního chatu.

## Co model vidí

Výběr projektu dá modelu přístup k danému adresáři prostřednictvím nástrojů. Neznamená to, že se obsah všech souborů vloží do kontextového okna. Model dostane kompaktní mapu projektu a jednotlivé soubory čte podle potřeby. Výjimkou jsou připnuté soubory a tři vrstvy paměti, které jsou záměrně vkládány jako trvalý kontext.

# 2. Požadavky a instalace

## Doporučená pracovní stanice

| Součást | Doporučená konfigurace |
|---|---|
| Operační systém | Windows 11, 64 bitů |
| GPU | NVIDIA RTX 5090 s 32 GB VRAM |
| Ovladač | Aktuální NVIDIA ovladač kompatibilní s přibaleným CUDA buildem |
| Systémová RAM | Dost pro Windows, mapování modelu, projekty a nástroje; komfortní je 64 GB a více |
| Volné místo | Podle vybraných modelů; Flash-Next IQ3_S potřebuje asi 92 GB (84,5 GB vah a zhruba 7 GB, které si připraví) |
| Python | Full: vlastní Python 3.12 přibalený. Minimal: 64bitový Python 3.12 instalovaný samostatně |
| Desktopové okno | WebView2 se připraví automaticky; Full obsahuje offline runtime |
| Prohlížečové nástroje | Microsoft Edge, běžně součást Windows 11 |

Jiné NVIDIA karty mohou fungovat, ale dodané kontexty a kvantizace byly nastaveny a ověřeny pro RTX 5090 s 32 GB. Karty s menší VRAM potřebují menší kontext, nižší kvantizaci, méně GPU vrstev nebo CPU offload.

POZNÁMKA: K dispozici jsou dva instalátory. **Minimal** stahuje balíčky a llama.cpp a vyžaduje samostatně nainstalovaný Python 3.12. **Full** obsahuje vlastní Python 3.12, uzamčené balíčky a llama.cpp/CUDA knihovny. Full vytvoří nové izolované venv místně, neregistruje Python a nemění systémový PATH. Obě varianty stahují modely samostatně; ovladač NVIDIA instaluje uživatel. Offline Backup zůstává samostatným balíčkem.

WebView2 se při instalaci automaticky rozpozná a připraví. Full obsahuje celý runtime; Minimal jej stáhne jen v případě potřeby. Existující kompatibilní runtime se použije beze změny. Pokud je WebView2 později odstraněn, Marvin jej znovu připraví při spuštění. Není potřeba navštěvovat stránky Microsoftu ani jej instalovat samostatně.

## Instalace pomocí Setup.exe

1. Pro soběstačné prostředí aplikace vyberte Full. Pouze pro Minimal nejprve nainstalujte 64bitový Python 3.12 a zapněte **Add Python to PATH**.
2. Spusťte `Marvin-Setup-<verze>.exe`.
3. Zvolte angličtinu nebo češtinu. Výchozí je angličtina.
4. Zvolte, zda chcete ikonu na ploše.
5. Při první instalaci ponechte zvolený setup prostředí a modelů po dokončení instalátoru.
6. Nechte konzolovou instalaci doběhnout. Vytvoří Python virtuální prostředí, nainstaluje závislosti, stáhne `llama.cpp` a nakonfigurované modely.
7. Spusťte **Marvin** z nabídky Start nebo z plochy.

Výchozí instalační adresář je:

```
%LOCALAPPDATA%\QwenHarness
```

Objem stahování závisí na zvolených modelech. Přerušené stahování lze obnovit opětovným spuštěním **Instalace prostředí a modelů** z nabídky Start. Volitelný Flash-Next se stahuje až při výběru v aplikaci nebo v seznamu modelů instalátoru.

## Qwen3.8-Flash-Next

V nastavení modelu vyberte **Qwen 3.8 Flash-Next · IQ3_S**, případně **Qwen 3.8 Flash-Next · IQ2_XS** s menšími vahami, které na počítačích se 48 nebo 32 GB RAM odpovídají rychleji. Při prvním výběru si Marvin připraví vše, co model potřebuje: jeho jádro, váhy (stáhne je a ověří včetně podpory obrázků), soubory, přes které váhy čte, a pomocnou vrstvu pro rychlejší odpovědi o velikosti asi 5 GB. Každá fáze ukazuje svůj průběh; přenos lze zastavit a později obnovit. Na testovacím počítači trvala celá první příprava asi 20 minut, většinu z toho stahování pomocné vrstvy; další starty trvají 30 sekund až minutu a půl.

Ovládání chatu, příloh a nástrojů je stejné jako u ostatních modelů a na Flash-Next i zpět můžete přepnout uprostřed chatu: konverzace, její soubory i dřívější odpovědi zůstávají.

Flash-Next drží většinu vah v systémové RAM a nejpoužívanější části na grafické kartě. Běží na kartách NVIDIA se 16 GB a více, na počítačích s 32 GB RAM a více a s ovladačem NVIDIA 580 nebo novějším. Marvin vybere profil změřený pro vaši kartu a vaši RAM; oba nabízejí kontext 256k a 128k. Na kartě se 48 nebo 96 GB použije stejné nastavení jako na 32GB kartě a větší grafickou paměť zaplní dalšími experty modelu, takže běží rychleji (odhad, neměřeno). Dlouhý nový dokument přečte zhruba za minutu; navazující otázky využívají již přečtený kontext.

## První spuštění

Marvin po spuštění automaticky načte naposledy úspěšně používaný model s jeho profilem KV cache. Platí to i při přímém spuštění webového rozhraní. Neúspěšné přepnutí nepřepíše poslední úspěšnou volbu. Tlačítko startu nebo restartu se okamžitě rozsvítí a během načítání jemně pulzuje. Panel modelu ukazuje fázi, uplynulý čas a využití paměti grafické karty. Stop zůstává během načítání dostupný.

Nahrávání příloh, export, změny nastavení, obnova a další požadavky mají okamžitou odezvu a následné potvrzení nebo chybu. Potvrzení znamená přijetí požadavku; zařazená práce modelu dále běží v přehledu konverzace. Zálohování ukazuje průběžný výpis i dokončení či selhání v sekci Data a zálohy.

Desktopový launcher kontroluje Python prostředí, závislosti, `llama.cpp` a potřebné soubory modelů. Chybějící části spustí instalační workflow. Po dokončení instalace se při běžném spouštění modely znovu nestahují.

Spuštění desktopové aplikace otevře nativní WebView okno, spustí Web UI a podle potřeby aktivuje `llama-server` s vybraným modelem. Zavření desktopové aplikace zastaví její Web UI i modelový server a uvolní grafickou paměť.

## Aktualizace

Nainstalujte nový Setup.exe přes existující instalaci. Uživatelská data, modely, relace, projekty, uživatelské skills a uložený stav UI zůstanou zachovány. Soubory aplikace a distribuované systémové skills se aktualizují. Model se znovu stahuje jen tehdy, když nakonfigurovaný soubor chybí.

## Odinstalace a odstranění všech dat

Odinstalace ve Windows odstraní nainstalované soubory aplikace. Velká runtime data a uživatelská data mohou zůstat, aby aktualizace nebo reinstalace nemusela vše znovu stahovat.

Chcete-li odstranit úplně vše, zkontrolujte a smažte zbývající instalační adresář pouze tehdy, když už nepotřebujete `runtime\models`, `sessions`, `projects`, `memory`, `user-skills` a `projects.json`.

VAROVÁNÍ: Smazání těchto adresářů je nevratné. Nejdříve zálohujte projekty, relace, paměť a uživatelské skills.

## Spuštění ze zdrojového checkoutu

```
py -3.12 -m venv .venv
.venv\Scripts\python scripts\setup_env.py --model auto
npm --prefix frontend ci
npm --prefix frontend run build
.venv\Scripts\python launcher\launcher_app.py
```

Pro terminál použijte `run_cli.bat` nebo `.venv\Scripts\python tui.py`.

Při práci ze zdrojů je potřeba Node.js pro sestavení frontendu. Běžná instalace Setup.exe už sestavený frontend obsahuje a Node.js nevyžaduje.

# 3. Prohlídka desktopového rozhraní

Vlevo je navigace projektů a chatů, uprostřed konverzace a vpravo zavíratelný panel detailů. Prompt zůstává dole pod konverzací. V menším okně se detail otevírá jako zásuvka.

Tato kapitola popisuje pracovní okno verze 1.8.0. Model a jeho stav najdete v horní liště. Pravý sloupec otevřete ikonou detailu nad konverzací nebo kliknutím na ukazatel kontextu pod zprávou. Obsah pravého sloupce se posouvá samostatně; jeho spodní patička **© Petr Sajner 2026** zůstává na místě. Stejný copyright je ve startovacím okně.

## První úloha krok za krokem

1. Klikněte na název modelu nahoře a zkontrolujte volbu v **Model a zařízení**. Pro běžnou práci ponechte automatickou detekci paměti. Počkejte na stav **Připraven**.
2. Vlevo vyberte projekt, připojte existující složku přes menu projektu nebo ponechte **Bez projektu**. Klikněte na **Nový chat**.
3. Vedle názvu chatu zvolte pracovní režim. Například **Psaní** pro dokument nebo **Vývoj** pro práci v repozitáři.
4. Napište požadovaný výsledek. Podle potřeby přidejte podklady tlačítkem **Attach**, přetažením nebo vložením ze schránky.
5. Odešlete zprávu. Průběh sledujte v konverzaci a v pravé záložce **Průběh**. Během práce lze upřesnit zadání, připravit další zprávu do fronty nebo úlohu zastavit.
6. Hotové soubory najdete v **Výsledky**. Otevřete náhled nebo jejich složku a ověřte výsledek. Běžná textová odpověď zůstává přímo v chatu.

## Navigace a pracovní režimy

Šířku navigace i pravého detailu změníte tažením za jejich vnitřní okraj. Dvojklik na okraj obnoví výchozí šířku; totéž provede restart nebo obnovení stránky. Seznam konverzací nejprve ukáže 20 nejnovějších chatů vybraného projektu. **Zobrazit starší** přidá dalších 20, **Jen nejnovější** seznam opět zkrátí.

Desktopový spouštěč používá data své vlastní instalace. Vývojová kopie a nainstalovaný Marvin neslučují konverzace. Pro výslovné otevření existujících dat z vývojové kopie slouží `python marvin_web.py --data-dir "C:\cesta\k\Marvinu"`. Data zůstávají na původním místě, bez převodu nebo kopírování.

Výběr projektu otevře jeho nejnovější chat. **Bez projektu** otevře samostatnou konverzaci. Přepnutí pohledu nepřesune ani nepřenastaví úlohu, která právě běží v jiném chatu.

Režimy mají vždy pořadí **Diskuze, Výzkum, Psaní, Vývoj, Počítač**. Režim patří konkrétnímu chatu. Probíhající požadavek si zachová konfiguraci, se kterou začal.

Tlačítko **Nový chat** vytvoří konverzaci. Název nad chatem přejmenujete zadáním textu a Enterem. Třítečkové menu chatu obsahuje přesun, vrácení posledního kola, větev, export a smazání. Výběr cílového projektu chat přesune a přepočítá jeho projektový kontext. Před přesunem nebo smazáním zastavte úlohu tohoto chatu.

Menu vedle výběru projektu vytvoří nový projekt, připojí existující složku nebo smaže projekt i jeho adresář. Vyhledávání vlevo prohledává uložené konverzace.

## Jak najít cokoli, co jste napsal

Pole v horní části panelu hledá **všude**, nejen v konverzacích: v chatech, v textových souborech otevřeného projektu, ve všech třech vrstvách paměti a v rozhodnutích projektu. Výsledky jsou rozdělené podle toho, kde leží, a kliknutím se otevře právě to místo — konverzace, dokument, stránka paměti, panel rozhodnutí.

Který projekt se prohledává, se řídí otevřenou konverzací. Bez projektu se hledá jen v konverzacích a v globální paměti, protože jinde není co hledat.

## Přejít na — jedna klávesa

**Ctrl+K** otevře malé pole, do kterého se píše. Nabídne, co rozhraní umí — nový chat, přepnutí režimu, katalog schopností, panely Kontext, Průběh a Změny, jednotlivé stránky nastavení, diktování, když je připravené — a pod tím to, co vaše slova našla ve vaší vlastní práci. Enter spustí první akci, Escape zavře.

Je to zkratka, ne samostatná funkce: vše, co nabízí, už v rozhraní někde je.

## Co si můžu říct?

Vedle tlačítka **Attach** je **Co si můžu říct?**. Otevře katalog zhruba dvaceti věcí, které Marvin umí, napsaných jako požadavky, ne jako seznam funkcí, a u každé je hotový příklad. **Použít** vloží příklad do promptu, kde ho lze upravit; samo o sobě nic neodešle.

Katalog ukazuje, co nabízí aktuální pracovní režim. Položka z jiného režimu je šedá, píše, který režim potřebuje, a tlačítkem do něj konverzaci přepne — pořádná rešerše je například schopnost režimu Výzkum, ne způsob formulace otázky. **Zobrazit vše** vypíše všechny bez ohledu na režim. Kde má schopnost vlastní místo v rozhraní — paměť, rozhodnutí, změny úlohy, zdroje výzkumu — položka na ně odkáže, místo aby je zdvojovala.

## Prompt, Attach a náhledy obrázků

Viditelné tlačítko **Attach** zůstává dole u promptu. Otevře výběr z disku a dovoluje vybrat více souborů. Obrázky lze také přetáhnout do pracovní plochy nebo vložit ze schránky přes Ctrl+V. Běžné vložení textu funguje dál.

Každý připravený obrázek má skutečný náhled a vlastní křížek pro odebrání. Odebráním jednoho obrázku nezmizí text ani ostatní přílohy. Kliknutí zvětší originál. Po odeslání jsou náhledy součástí uživatelské zprávy a zůstanou dostupné i po opětovném otevření chatu.

Přiložit lze také PDF, DOCX, XLSX, CSV, Markdown a text. Model dostane cesty k dokumentům a přečte je příslušnými nástroji. Textový model neumí obsah obrázku; pro obrazové vstupy zvolte model s vision. Při odmítnutém požadavku zůstanou text i přílohy v draftu.

Enter a Ctrl+Enter odešlou zprávu, Shift+Enter vloží nový řádek. Composer se vyčistí po přijetí identifikované zprávy službou. Rozepsaný text se ukládá pro každý chat zvlášť.

### Navržený další krok

Během konverzace si model průběžně skládá představu, o co vás pravděpodobně požádáte dál: k čemu směřujete, co se vyřešilo a co zůstává otevřené. Když chat nepracuje a pole pro zprávu je prázdné, ukáže se v něm tato představa šedým textem. Klikněte do pole a stiskněte šipku doprava (→) nebo Tab: z návrhu se stane skutečný text, který můžete upravit, nebo rovnou odeslat. Cokoli jiného, co napíšete, návrh nahradí. Když model žádný návrh nedá, dokončená úloha nabídne, co podle jejího stavu zůstává otevřené: neprošlou kontrolu, nedokončený krok plánu nebo první z dalších kroků v závěrečném shrnutí. Návrh je součástí běžných odpovědí modelu, takže nestojí žádný dotaz navíc a nic nezpomaluje. Vypnete ho v **Nastavení > Chování > Navrhovat další krok**; model ho pak přestane skládat a existující konverzace při dalším požadavku jednou znovu přečte svůj prompt.

## Diktování

V **Nastavení > Chování** zapněte **Hlasový vstup**. Poprvé se na pozadí stáhne asi 556 MB: program pro rozpoznávání řeči a model, který umí česky. Pak vše běží na procesoru, v tomto počítači, a nic se nikam neposílá.

Vedle Attach se objeví tlačítko **Diktovat**. Stisknete, mluvíte, stisknete znovu. Text se zapíše do pole promptu, kde si ho přečtete a opravíte, než odešlete. Samotným hlasem se nikdy nic neodešle.

V **Jazyk diktování** uveďte, jakým jazykem mluvíte; ponechat jazyk rozhraní je běžné. Uvedený jazyk je zhruba dvakrát rychlejší než automatické rozpoznání, protože rozpoznání jazyka je samostatný průchod nahrávkou.

Co čekat: běžná čeština se vrací čistá. Chyby dělají jména, cizí slova a odborné termíny — proto text končí v poli k úpravě a neodesílá se rovnou. Přepis trvá kolem čtyř sekund, ať mluvíte dvě sekundy nebo deset.

Když tlačítko stisknete a nic neřeknete, dostanete „Nic jsem neslyšel" místo věty, kterou si program vymyslel — ticho se rozpozná ještě před přepisem.

## Generování obrázků

Všechno ostatní v Marvinovi běží na vašem počítači. Tahle jediná funkce ne: obrázky generuje přes OpenArt, na vašem účtu OpenArt, a každý obrázek stojí kredity. Dokud ji nezapnete, je vypnutá.

V **Nastavení > Chování** zapněte **Povolit placené generování obrázků**. Jednorázově se stáhne 4MB program. Pak stiskněte **Přihlásit k OpenArt**: přihlášení se otevře v prohlížeči a dokončíte ho tam. Marvin vaše heslo nikdy nevidí a žádný credential neukládá — program OpenArt si drží vlastní ve vašem uživatelském profilu a nic z něj se nedostane do nastavení Marvina, exportu ani zálohy. Není co vkládat jako API klíč, protože OpenArt žádný nepoužívá.

Jakmile je účet připojený, svítí zvýrazněný štítek se zůstatkem kreditů. **Přepínač je nezávislý na přihlášení**: když ho vypnete, generování se zastaví, i když účet zůstane připojený.

Model pak umí vygenerovat obrázek, když je opravdu potřeba — sprite, texturu, návrh — a ukládá ho do složky `generated-images` v aktuálním projektu, kde ho otevřete jako každý jiný soubor. Zná pět modelů: Google Nano Banana 2 (výchozí, 4K s přesným textem v obrázku), OpenAI GPT Image 2.5 ve verzi rychlejší a přesnější, Nano Banana Pro na dlouhý text a konzistentní postavy a Seedream 4.5 na anime a ilustraci.

Co čekat: jeden obrázek zhruba sedmnáct sekund a cena se napíše k výsledku vedle souboru. Obrázek z Nano Banana 2 stál v době psaní 20 kreditů. Marvin utrácení nestropuje; ovládáte ho tím přepínačem.

Bez připojení k internetu je tohle jediná věc, která fungovat nemůže. Řekne to a nic jiného to neovlivní.

## Běžící úloha a fronta zpráv

Během práce zvolte **Upřesnit nyní** pro steering nebo **Po dokončení** pro samostatný další požadavek. Úloha v jiném chatu pokračuje a nová zpráva čeká na jediný modelový worker. Text ve frontě lze upravit a čekající zprávu zrušit.

**Zastavit úlohu** funguje při zpracování promptu, přemýšlení, psaní, přípravě nástroje i synchronní akci. U viditelného textu může krátce dokončit větu; nečeká na dokončení celého reasoning bloku. Hotová práce i částečný

text zůstanou zachované. Stop pozastaví také čekající frontu; **Spustit zprávy ve frontě** ji opět spustí. **Pokračovat** obnoví přerušenou úlohu s právě vybraným modelem a jeho aktuálním KV profilem. Po ručním přepnutí se již nenačítá původní model úlohy. Historie a rozpracovaná úloha zůstávají zachované; menší kontext používá běžnou kompresi historie.

Reload a obnovení spojení prohlížeče neruší běžící modelovou úlohu. Po restartu celé aplikace lze přerušenou práci obnovit. Když před pádem nebyl uložen výsledek nástroje, model dostane informaci, aby ověřil skutečný stav místo slepého opakování akce.

## Výsledky, Průběh a Kontext

**Výsledky** ukazují vytvořené dokumenty a změněné soubory, skutečně zaznamenané kontroly, výzkumné exporty a body obnovy. Soubor můžete prohlédnout, otevřít jeho složku nebo stáhnout. HTML výsledek lze zobrazit jako aplikaci a podporované programy mají také akci **Spustit**.

**Průběh** spojuje plán, trvale uložené provozní události, změny souborů, procesy na pozadí s výstupem a stav izolovaného browseru. Samostatné procesy podle potřeby zastavujete zvlášť.

**Kontext** zobrazuje odhad obsazení a fyzický limit, naměřenou spotřebu posledního dokončeného požadavku, pokud je dostupná, tři paměťové vrstvy, pins a načtené skilly. Můžete připnout či odepnout jednotlivé soubory, zrušit všechny pins, komprimovat kontext nebo předat práci novému chatu.

Při generování stavový řádek rozlišuje přípravu, uvažování, psaní odpovědi, přípravu nástroje a provádění. Ukazuje také uplynulý čas a výslovně přibližný počet právě generovaných tokenů.

## Nastavení

Klikněte na ikonu nastavení nebo stav modelu nahoře. Nastavení je rozdělené do kategorií:

- **Model a zařízení**: model, KV profil, vision, paměť GPU, provozní diagnostika a Start / Stop / Restart.
- **Chování**: autonomie a výchozí zacházení se zprávou přidanou během práce.
- **Paměť a skilly**: úprava globální, režimové a projektové paměti; čtení, použití a návrh skillu; otevření uživatelské i projektové složky skillů.
- **Data a zálohy**: volba složky pro nové projekty, export/import celého projektu, import JSONL chatu, vytvoření/výběr/kontrola lokální zálohy prostředí a stav údržby.
- **Vzhled a jazyk**: tmavý, světlý nebo systémový vzhled, rozestupy, angličtina a čeština.
- **Nápověda a manuály**: oba PDF manuály a reference lomítkových příkazů.

Myšlení zůstává také u promptu, protože ho můžete měnit mezi otázkami. Nastavení zvolené za běhu platí pro následující požadavky. Přepnutí jazyka a vzhledu nezahazuje konverzaci.

## Projektová rozhodnutí

**Přijatá rozhodnutí** evidují navržená, přijatá i neplatná rozhodnutí. Rozhodnutí lze přidat, upravit, změnit jeho stav a otevřít původní konverzaci. Do projektového kontextu automaticky vstupují jen přijaté položky. Doplňují tři paměti; návrh se sám nemění na závazný fakt.

# 4. Modely, KV cache, kontext a thinking

| Třída GPU | Model | Přesnost KV | Dostupné kontexty |
|---|---|---|---|
| 16 GB | Qwen IQ3 | Q8 | 64k / 48k |
| 24 GB | Qwen IQ3 | Q8 | 128k / 96k |
| 24 GB | Qwen Q4 | Q8 | 96k / 64k |
| 24 GB | Qwen Q5 | Q8 | 96k / 64k |
| 24 GB | Nemotron Q4 | Q8 | 512k / 256k |
| 24 GB | Nemotron Q5 | Q8 | 512k / 256k |
| 32 GB | Qwen Q4 | Q8 | 256k / 192k |
| 32 GB | Qwen Q4 | F16 | 128k / 96k |
| 32 GB | Qwen Q5 | Q8 | 192k / 128k |
| 32 GB | Qwen Q5 | F16 | 128k / 96k |
| 32 GB | Ornith Q5 | Q8 | 256k / 192k |
| 32 GB | Nemotron Q4 | Q8 | 512k / 256k |
| 32 GB | Nemotron Q5 | Q8 | 512k / 256k |

Na kartách třídy 32 GB je výchozí Qwen Q5 s Q8 KV a kontextem 192k. Menší karty dostanou odpovídající Q8 profil podle tabulky. F16 je volitelně pouze pro Qwen Q4/Q5 na 32GB kartách. IQ3 se nabízí pouze na 16/24GB kartách. Každá kombinace modelu a přesnosti KV nabízí dvě nejvyšší odpovídající velikosti kontextu.

Karty se 48 GB a více (včetně 64 a 72GB karet) používají vlastní profily **48 GB** a karty s 96 GB profily **96 GB**. Jsou odhadnuté ze změřených hodnot pro 32 GB, nikoli změřené, protože taková karta nebyla k dispozici: vše, co se vejde na 32GB kartu, a navíc Qwen Q4/Q5 s Q8 256k (i s MTP) a F16 256k a Nemotron Q5 s 512k celý na GPU. Qwen a Ornith zůstávají nejvýš na 256k, Nemotron na 512k.

Qwen Q4/Q5 nabízejí také volitelné **MTP varianty** svých Q8 profilů (například „Q8 · 192k · MTP" vedle „Q8 · 192k"). MTP varianta generuje zhruba 2–3× rychleji pomocí malého draft modelu trénovaného pro stejný model; kvalita odpovědí zůstává stejná. Potřebuje asi 2–2,9 GB dodatečné paměti GPU a 1,4 GB draft model se při prvním spuštění automaticky stáhne a ověří. Na kartách třídy 24 GB mají MTP varianty pouze Qwen Q4. Pokud při běhu MTP relace dojde paměť, Marvin nejprve přepne na stejný kontext bez MTP, poté zkusí nižší kontext s MTP a nakonec bez MTP.

Když běží spekulativní profil, záhlaví zobrazuje odznak **MTP** vedle názvu modelu. Stavový řádek generování pod konverzací ukazuje aktuální rychlost (tokeny za sekundu) spolu s odhadem tokenů, takže rozdíl mezi plain a MTP profily je při práci vidět.

**Sémantické vyhledávání** (Nastavení > Model a zařízení) přidává hledání podle významu v projektových souborech i minulých konverzacích — česky i anglicky, včetně parafrází — kombinované s klíčovým hledáním. Ve výchozím stavu je vypnuté; zapnutí stáhne malý embedding model (~635 MB), který běží jen na CPU a grafickou kartu vůbec nevyužívá. První hledání v projektu staví index na pozadí; agent vidí průběh a hledání podle potřeby zopakuje.

Q5 používejte pro náročný vývoj, architekturu a finální kvalitu. Q4 je vhodný pro vyšší rychlost nebo kontext 256k. Ornith je extrémně rychlý, ale při reálném vývoji může být slabší než dense Qwen.

Nemotron je pouze textový. Qwen, Ornith a Flash-Next mají vlastní podporu obrazových vstupů. Nabídka modelů ukazuje dostupnost souborů; Flash-Next potřebuje své shardy, projektor a připravené soubory a chybějící si Marvin připraví při jeho výběru. Podle karty jsou u Qwenu dostupné také kompaktní profily. Tabulka není příslibem, že každá kombinace poběží na každém počítači; přesný výběr najdete u zvoleného modelu v nastavení.

## Limit paměti GPU a automatická obnova

Pro běžnou práci ponechte **Nastavení > Model a zařízení > Limit paměti GPU** na **Automatická detekce**. Ruční hodnota omezuje paměť dostupnou Marvinovi; nezvětší skutečnou kapacitu karty. Měňte ji, když neběží úloha. Marvin uvolní předchozí model, vybere vhodný profil a podle potřeby model restartuje. Úspěšný model a kontext si pamatuje pro každý limit, takže návrat k automatické detekci obnoví její nastavení. Pokud změna selže, pokusí se vrátit předchozí funkční model i nastavení.

Qwen IQ3 používá na 16 GB Q8 64k/48k, Qwen Q5 na 24 GB Q8 96k/64k. Oba zachovávají vision se zpracováním obrázků na CPU. Nemotron Q4/Q5 nabízí na 24 GB 512k/256k s částí expertů na CPU. Také Nemotron Q5 na 32 GB při 512k používá jeden blok expertů na CPU; při 256k jsou váhy na GPU. Tato rozložení byla změřena na RTX 5090, nikoli na fyzických 16/24GB kartách. Jiné programy mohou ovlivnit, zda se větší volba úspěšně spustí.

Pokud skutečná chyba alokace nebo trvající kritický nedostatek fyzické RAM přeruší úlohu, Marvin zkusí nejbližší menší kontext se stejným modelem, přesností cache i rozložením vah. Výsledky dokončených nástrojů zůstanou v historii. Obnova nepřepíná mezi Q8 a F16. Relace spuštěná s variantou MTP nejprve zruší draft na stejném kontextu a poté postupně zkouší nižší kontexty s MTP i bez něj. Pokud už menší podporovaný kontext nezbývá, úloha se zastaví s vysvětlením; můžete vybrat jiný model a použít **Pokračovat**. Úspěšné nastavení se pamatuje pro každý limit GPU.

## Flash-Next: nastavení a reálné čekání

**IQ3_S** stáhne 84,5 GB ve třech souborech. **IQ2_XS** stáhne 69 GB, z toho 29,7 GB (druhý shard a projektor obrázků) jsou tytéž soubory jako u IQ3_S, takže vedle IQ3_S přidá asi 39 GB. První příprava přidá zhruba 7 GB souborů modelu a 0,7 GB jádra. Další starty používají místní kopie; změna kontextu nic z toho neopakuje.

Načtení trvá asi 30 sekund s 64 GB RAM a na menších počítačích, které část vah čtou z SSD, až kolem minuty a půl. Zatímco model plní systémovou RAM, může počítač chvíli reagovat pomaleji.

Obě varianty vah používají Q8 cache s profily 256k a 128k. Marvin vybírá podle karty (32, 24 nebo 16 GB) a RAM (64, 48 nebo 32 GB; více než 64 GB používá profily pro 64 GB). Změřeno v říjnu 2026 na RTX 5090 se simulovanými menšími kartami a RAM; *odpověď* je krátká odpověď bez thinking, *dlouhý vstup* první odpověď na vstup, který zaplní 256k:

| IQ3_S | karta 32 GB | karta 24 GB | karta 16 GB |
|---|---|---|---|
| 64 GB RAM | 110 tokenů/s, dlouhý vstup 43 s | 92 tokenů/s, 43 s | 73 tokenů/s, 49 s |
| 48 GB RAM | 72 tokenů/s, 56 s | 41 tokenů/s, 71 s | 25 tokenů/s, 4,3 min |
| 32 GB RAM | 45 tokenů/s, 2,9 min | 24 tokenů/s, 4,7 min | 13 tokenů/s, 10 min |

IQ2_XS odpovídá rychlostí 107–136 tokenů/s s 64 nebo 48 GB RAM na všech kartách a s 32 GB RAM rychlostí 119 / 71 / 32 tokenů/s na kartě 32 / 24 / 16 GB. Navazující dotaz z cache se všude vrátí zhruba za sekundu. Pro srovnání: předchozí verze Flash-Next odpovídala asi 25 tokeny/s a na stejně dlouhý vstup potřebovala na nejsilnějším počítači asi 44 minut.

Když během úlohy dojde paměť, Marvin spustí stejný model znovu na 128k se stejným nastavením a pokračuje; výsledky dokončených nástrojů zůstanou v historii. Se 48 GB RAM a 16GB kartou může velmi dlouhý vstup přesáhnout paměť i na 128k; pak vyberte menší model a použijte **Pokračovat**.

Profily používají tradiční označení 128k/192k/256k pro 131 072 / 196 608 / 262 144 tokenů. Číselný ukazatel může tutéž kapacitu zaokrouhlit například na 262k.

## Přesnost KV

Kvantizace vah a KV cache jsou dvě různé věci: **IQ3_S** u Flash-Next popisuje váhy modelu, **Q8** jeho průběžnou paměť kontextu. F16 používá více paměti pro KV, Q8 ji šetří. Výslednou velikost okna určuje konkrétní profil a dostupná paměť. Změna KV restartuje server, nikoli chat.

### Thinking

U Qwenu jsou `xhigh`, `medium`, `low` nativní reasoning effort a `off` thinking vypíná. U Ornithu je nativní pouze on/off: `xhigh` a `medium` jsou promptové vodítko, `low` běžné thinking chování a `off` vypnutí.

Reasoning tokeny používají čas i kontext; UI je živě odhaduje. Uložené uvažování lze rozbalit u odpovědi. Odhad persistentního kontextu zahrnuje i uvažování předávané zpět modelu.

Produkční agent nemá limit kroků ani aplikační limit výstupních tokenů.

# 5. Odesílání, steering, Stop a průběh

## Prompt a přílohy

Pište přirozeným jazykem a uveďte požadovaný výsledek, omezení a způsob ověření. Pomocí **Attach**, přetažení nebo Ctrl+V lze přidat BMP, GIF, JPEG, PNG nebo WebP: fotografie, diagramy, screenshoty, UI reference i chyby.

## Steering

Při volbě **Upřesnit nyní** další zpráva v právě běžícím chatu přesměruje aktivní úlohu. Hotový text zůstane uložen a práce pokračuje s upřesněním. **Po dokončení** vytvoří samostatný následující požadavek. Zpráva do jiného chatu čeká ve frontě na jediný model. Zpráva doručená těsně při dokončování se neztratí a přejde do čekající práce.

## Stop

**Stop** obchází běžnou frontu a měkce ukončí generování po nejbližší větě. Současně zruší právě čekající browser operaci nebo synchronní `run_command` a ukončí jeho procesní strom. Samostatný proces spuštěný na pozadí zastavte zvlášť v **Průběh > Procesy**.

Rozlišujte **Zastavit úlohu** u zprávy a **stop** v nastavení modelu. První ukončí aktuální práci a pozastaví frontu, ale model může zůstat připravený v paměti. Druhé zastaví modelový server a uvolní jeho prostředky. Stažené modely zůstávají na disku pro další spuštění.

Tlačítka v **Průběh > Procesy** se jmenují **Zastavit tento příkaz** a ukončí jen ten jeden příkaz na pozadí — úloha běží dál. Úlohu ukončíte tlačítkem u promptu.

**Přerušení během čtení kontextu.** Velký prompt se přečte jednou a pak se používá znovu, takže každý další krok dopočítá jen nový kousek. Zkrátit to čtení znamená zahodit všechno už přečtené a příští požadavek za to zaplatí znovu — u opravdu velkého kontextu jsou to minuty. Připomínka proto počká, až se kontext dočte, a uplatní se na odpověď hned potom; konverzace to po dobu čekání oznámí. **Stop** čekat nenechává, protože je to výslovný pokyn — ale po zastavení během čtení kontextu začne další zpráva to čtení od začátku.

## Když něco selže

Selhaná úloha po sobě nechá zprávu v **Průběh > Historie aktivit**, ne jen oznámení, které zmizí. Ukáže, co selhalo, a tam, kde Marvin příčinu pozná, i co s tím: projekt bez vlastního virtuálního prostředí, model, který se nespustil, žádný model do limitu paměti, přesunutá složka projektu nebo došlá paměť.

Kde odpověď známa není, nic se nevymýšlí. **Zjistit, co s tím** vloží do promptu připravenou otázku i s chybou a předá ji modelu, který na příčinu obvykle přijde. Otázka se sama neodešle, takže si ji můžete přečíst a upravit.

Totéž nabídne chybný výsledek nástroje v konverzaci i kontrola projektu, která selhala, vypršela nebo se vůbec nespustila.

## Živý průběh

Zatímco model čte kontext, řádek ukazuje, kolik konverzace měl server ještě z předchozího požadavku - podíl z celého promptu - a odhad zbývajícího času podle právě měřené rychlosti.

Vysoký podíl je normální stav: harness požadavek jen doplňuje na konci, takže model nic nečte znovu. Nízký podíl znamená, že se změnilo něco blízko začátku konverzace a zpracovává se celá. Záměrně to dělají dvě věci - komprese a vzdání se starších snímků obrazovky, když se kontext plní. Obojí se v chatu ohlásí.

UI rozlišuje reasoning, viditelný text, přípravu velkých tool callů, provádění nástrojů/příkazů/testů a sekundy bez tokenů. Při tvorbě velkého souboru ukazuje jméno souboru a rostoucí množství připravovaného obsahu.

# 6. Pracovní režimy

| Režim | Použití |
|---|---|
| Diskuze | Běžný chat, analýza, učení a brainstorming bez coding procedur. |
| Výzkum | Web/dokumentové rešerše, persistentní ledger a závěrečná syntéza. |
| Psaní | Scénáře, články, reporty, textové revize a export dokumentů. |
| Vývoj | Repozitář, soubory, Git, shell, testy, checkpoint a rollback. |
| Počítač | Vývoj plus screenshot, myš, klávesnice a scrolling. |

Každý chat si pamatuje vlastní režim. Režimy jsou profily schopností stejného modelu, ne samostatní agenti.

# 7. Projekty a workspace

## Vytvoření a připojení

V třítečkovém menu vedle projektu vytvořte spravovaný projekt nebo pomocí **Připojit existující složku** zaregistrujte adresář bez kopírování.

Projekt může mít libovolný počet chatů. Každý má vlastní historii, režim, kompresi, pins, research a task state; sdílejí složku a projektovou paměť.

## Kde se zakládají nové projekty

V **Nastavení > Data a zálohy** zvolíte složku pro projekty zakládané podle jména. Dokud není nastavená, vznikají vedle instalace. Složka musí existovat, být zapisovatelná a ležet mimo modelový runtime i historii konverzací; cokoli jiného se odmítne s uvedeným důvodem.

Změna nic nepřesouvá. Už existující projekty si svou složku nechají a dál z ní fungují — nové místo platí až pro další projekt.

## Bez projektu a přesun chatu

**Bez projektu** používá samostatný chat; exporty jdou do relace. V třítečkovém menu chatu zvolte cílový projekt. Přesun přenastaví workspace, projektovou paměť a knihovnu dokumentů.

## Smazání projektu

Klikněte **Smazat projekt**, ověřte zobrazenou cestu a potvrďte v dialogu. U projektu vytvořeného v Marvinu se smaže registrace, všechny chaty projektu, složka i její obsah. U projektu připojeného z vlastní složky se smaže jen registrace a chaty projektu — **vaše složka a její obsah vždy zůstávají na disku**. Pokud by složku nešlo smazat, nic se neodebere a zobrazí se srozumitelná chyba.

# 8. Správa chatů

- **Nový chat** vytvoří transientní relaci, která se uloží po první zprávě.
- **Smazat chat** vyžaduje potvrzení v dialogu a odstraní relaci včetně příloh, research a session exportů; projektové soubory zůstanou.
- Přejmenování se potvrzuje Enterem.
- **Znovu** zopakuje odpověď na poslední prompt.
- **Vrátit** odstraní poslední konverzační kolo, nikoli souborové změny.
- **Větev** vytvoří kopii chatu u posledního promptu včetně projektu, režimu, obrázků a pins.

- **Hledat ve všech chatech** používá SQLite fulltextový index.
- Export podporuje čitelný Markdown a strukturální JSONL; JSONL se importuje jako nový chat.
- **Předat chatu** vytvoří nový chat se souhrnem cílů, rozhodnutí, stavu a dalších kroků.

# 9. Kontext, komprese a pins

Panel **Kontext** ukazuje odhad tokenů, viditelné/celkové zprávy, obrázky, stav komprese a pins. Rozpad zvlášť uvádí konverzaci/přílohy, aktuální projektový kontext a definice nástrojů. Celkový údaj proto zahrnuje i režii tool schemas.

Ve Vývoji model dostává kompaktní repo snapshot; ostatní režimy katalog dokumentů. Snapshot, pins a aktivní projektové instrukce se skládají jen pro aktuální request a neukládají se jako stále nové kopie do historie.

Harness automaticky hledá `AGENTS.md`, `QWEN.md` a `CLAUDE.md` od kořene projektu po adresář právě čteného nebo měněného souboru. Hlubší dokument je konkrétnější. Jde o guidance; aktuální zadání uživatele má přednost.

**Připnout soubor** vloží text vybraného souboru do následujících úloh tohoto chatu. Vhodné jsou architektura, specifikace, handoff a projektová pravidla. Limit je 10 souborů a asi 40 000 znaků. Větev pins kopíruje, nový chat ne.

Při 85 % kontextu se nejdřív přestanou posílat všechny snímky obrazovky kromě čtyř nejnovějších. V režimu Počítač jsou zdaleka největší částí konverzace, takže to obvykle stačí a text konverzace zůstane celý. Obrázky zůstanou vidět v chatu; jen je model u starších zpráv přestane dostávat, a to už trvale.

Když to nestačí, starší část se automaticky shrne. Viditelná historie se nemaže. Ruční komprese je v **Kontext > Komprimovat**. Při overflow agent jednou automaticky komprimuje a request zopakuje.

# 10. Tři vrstvy paměti

| Vrstva | Rozsah | Umístění |
|---|---|---|
| Globální | Všechny režimy/projekty | `memory\GLOBAL.md` |
| Režimová | Všechny chaty daného režimu | `memory\MEMORY.md` nebo `memory\modes\*.md` |
| Projektová | Jeden workspace | `<projekt>\QWEN_MEMORY.md` |

Paměti otevřete v **Nastavení > Paměť a skilly** nebo řekněte modelu "zapamatuj si globálně", "pro Výzkum" či "pro tento projekt". Přesun chatu přepne projektovou paměť.

# 11. Volitelné skills

Skills jsou pomocné `SKILL.md` postupy. Nespouštějí druhý model a nepřebíjejí explicitní zadání. Model vidí jen jméno a trigger popis; celé tělo načte přes `read_skill` podle potřeby.

Priorita je projektový `.qwen-skills`, potom `user-skills`, potom distribuované `skills`.

| Skill | Účel |
|---|---|
| `systematic-debugging` | Reprodukce, trasování, hypotézy a příčina chyby. |
| `architecture-options` | Varianty architektury se zachováním zadání. |
| `implementation-verification` | Přiměřená kontrola výsledku, testů a releasu. |
| `performance-investigation` | Měřením řízená analýza výkonu. |
| `research-synthesis` | Úplná syntéza se zachováním rozporů. |

Vlastní skill vytvořte přes **Nastavení > Paměť a skilly > Uživatelské skilly** jako `user-skills\nazev\SKILL.md`:

```
---
name: muj-skill
description: S čím pomáhá a kdy ho má model načíst.
---

# Můj skill

Flexibilní doporučení a reference.
```

# 12. Internet a výzkum

## Běžná práce s webem

Všechny režimy mohou používat `web_search` a `web_fetch` pro aktuální informace, veřejnou dokumentaci, chybové zprávy a webové stránky. `web_fetch` zpracuje HTML a umí extrahovat text z podporovaných PDF a DOCX downloadů.

U dlouhých stránek může model vyhledat konkrétní výraz nebo načíst další pasáž bez opakovaného stahování. Celý načtený zdroj zůstává v evidenci výzkumu v rámci jejího technického limitu; užší výřez jej nenahrazuje. To pomáhá zvlášť u velkých modelů, které část výpočtu provádějí v systémové RAM.

Během načítání vstupu se zobrazuje **Načítám kontext**, průběh nového textu a počet tokenů použitých z cache. Jde o jinou fázi než generování přemýšlení a odpovědi. První dlouhý dokument může být pomalý, i když navazující otázka využije téměř celou historii z cache. Úroveň přemýšlení a zvolená kvantizace se tímto zrychlením nemění.

Procento na tomto řádku je označené jako **nové**, protože jde o postup tou částí promptu, kterou server modelu ještě nemá — ne o zaplněnost kontextu. Zaplněnost je vedle vstupního pole, jako `Kontext: ~132k / 197k`. Mezi požadavky je to odhad a sám se opravuje: server s každou odpovědí hlásí přesnou délku promptu, takže se odhad proti němu měří a upravuje. Nová konverzace začíná od změřených průměrů a postupně se zpřesňuje, protože česká próza, programový kód a snímky obrazovky stojí různý počet tokenů na znak.

Výchozí vyhledávač je Google; v `config.yaml` lze zvolit Google, Bing nebo automatický fallback. Fetching je read-only HTTP/HTTPS.

## Výzkumný workflow

Režim Výzkum automaticky:

1. Zaznamená otázku.
2. Před hledáním vytvoří trvalý research plán.
3. Uloží každý dotaz a kandidátní odkaz.
4. Uloží plný čitelný obsah webových i lokálních zdrojů.
5. Sleduje coverage a ID zdrojů.
6. Zachová rozpory a menšinové informace.
7. Vytvoří závěrečnou lidsky čitelnou syntézu.

Zdroje se neskrývají ani nevyhazují podle toho, zda je model považuje za důvěryhodné. Důvěryhodnost a relevanci posuzuje uživatel. Syntéza odlišuje tvrzení zdrojů od inference a uvádí nejistoty a chybějící důkazy.

## Panel výzkumu

V pravé záložce **Výsledky** otevřete sekci **Výzkum**. **Všechny zdroje** zobrazí podklady aktuální úlohy; tlačítka **PDF**, **DOCX** a **Zdroje** exportují hotovou syntézu nebo její zdrojový záznam. Obecný plán a provozní události jsou v záložce **Průběh**. Export již hotové odpovědi nespouští nový výzkum.

Podporované projektové dokumenty zahrnují Markdown, text, RST, CSV, JSON, YAML, PDF a DOCX. Když Výzkum přečte lokální dokument, zaznamená jej jako zdroj.

# 13. Psaní a export dokumentů

Režim Psaní zachovává záměr, hlas, strukturu a omezení uživatele. Umí číst a upravovat projektové soubory bez zavádění vývojové terminologie, pokud ji uživatel nepožaduje.

Model může v každém režimu exportovat finální text přirozeným požadavkem:

```
Ulož finální odpověď jako PDF.
Vyexportuj to jako strukturovaný DOCX s názvem production-report.
Vytvoř Markdown soubor s touto syntézou.
```

Podporovány jsou Markdown, strukturovaný DOCX a PDF s Unicode/českým textem, základními tabulkami a inline formátováním.

S projektem se výstup ukládá do `<projekt>\exports`. Bez projektu jde do `sessions\<id-chatu>\exports`. Projektové exporty mohou být součástí task checkpointu a lze je podle situace rollbackovat.

# 14. Vývojový workflow

## Poznání projektu

Při každém requestu dostane model aktuální repo snapshot a platné hierarchické projektové instrukce. Může použít `repo_overview`, `project_instructions`, procházet/globovat soubory, hledat literal nebo regex, navigovat deklarace přes `find_symbol`/`document_symbols`, hledat použití přes `find_references` a číst relevantní rozsahy.

## Souborové operace

- `read_file` čte text s čísly řádků.
- `search_files` používá ripgrep, podporuje literal/regex, case sensitivity a glob názvu.
- `find_files` vrací soubory odpovídající globu.
- `write_file` vytvoří nebo přepíše celý soubor.
- `apply_patch` provádí přesné atomické náhrady.
- `make_directory`, `move_file`, `delete_file` jsou strukturované operace zahrnuté do rollbacku.
- `view_image` vloží obrázek do vision kontextu.

## Izolovaný browser workflow

Vývoj a Počítač obsahují persistentní headless Microsoft Edge session oddělenou od běžného browser okna uživatele:

1. `browser_open` načte lokální nebo veřejnou URL.
2. `browser_snapshot` vrátí viditelný text a interaktivní prvky s refs jako `e1`.
3. `browser_fill`, `browser_click`, `browser_press` pracují přes tyto refs.
4. `browser_select`, `browser_hover`, `browser_scroll` obslouží dropdowny, hover a dlouhé stránky.
5. `browser_upload` nastaví lokální file input; `browser_download` uloží download k aktivnímu chatu.
6. `browser_viewport` přepne desktop, tablet nebo mobilní rozměr.
7. Nový snapshot ověří DOM výsledek; zastaralé refs jsou odmítnuty.
8. `browser_console` a `browser_network` vracejí konzoli, HTTP responses a selhané requesty.
9. `browser_screenshot` vloží vykreslenou stránku do dalšího requestu pro Qwen vision.
10. `browser_close` nebo **Browser session > Zavřít browser** uvolní Edge proces.

Browser přežije jednotlivé agentní kroky. Režim Počítač použijte pro nativní desktopové aplikace nebo celou obrazovku Windows.

## Plán úlohy

U větší práce model vytvoří operační plán pomocí `set_task_plan` a aktualizuje kroky přes `update_task_step`. Panel **Průběh úlohy** ukazuje cíl, pending/in-progress/completed kroky, poslední validaci a kontrolu diffu. Ledger je v `task-plan.json`, přežije restart i kompresi a neobsahuje soukromý chain-of-thought.

## Checkpoint a rollback

První zápis aktivuje change journal. **Změny této úlohy** vypíší vytvořené/upravené soubory. **Vrátit změny této úlohy** obnoví journalované soubory do stavu před úlohou a odstraní soubory, které úloha vytvořila.

Chatové **Vrátit** mění konverzaci. Task rollback mění souborový systém. Jde o dvě odlišné funkce.

## Git

Model umí číst status a diff a vytvořit lokální commit. `git_commit` staguje uvedené cesty nebo jen soubory aktuálního task journalu. Automaticky nepushuje. Commit i push požadujte výslovně.

## Příkazy a testy

Krátké příkazy běží synchronně s timeoutem. Úplný stdout/stderr se ukládá do `sessions\<id-chatu>\command-logs`; model dostane začátek i konec, takže neztratí závěrečnou chybu. Stop aktivní synchronní příkaz ihned ukončí. Dlouhé příkazy na pozadí vrátí process ID a lze je pollovat, posílat jim stdin nebo ukončit celý strom procesů.

`project_validation_profile` vypíše detekované test/lint/typecheck/build příkazy. `start_project_check` spustí primární nebo pojmenovanou kontrolu. Detekce pokrývá tento harness, složku `tests/` i moduly `test_*.py` v kořeni projektu, Node scripts, Rust, Go a .NET. Ve výchozím stavu se používá standardní `unittest`; pytest jen tehdy, když si ho projekt nakonfiguruje (`pytest.ini`, `conftest.py`, `[tool.pytest]`) a interpret ho umí naimportovat.

Projekt může detekci nahradit souborem `.qwen/project.yaml`:

```
checks:
  - id: tests
    label: Kompletní testy
    command: npm test
    shell: powershell
    kind: test
    timeout: 900
    primary: true
```

Kontroly běží ve vlastním virtuálním prostředí projektu (`.venv` nebo `venv`), pokud ho projekt má, takže vidí jeho závislosti. Bez něj je spustí interpret Marvina a chybějící cizí závislost se ohlásí jako důvod selhání. Lint a typovou kontrolu nabídne jen tehdy, když je nástroj nainstalovaný — detekovaná kontrola tedy neselže už při startu.

**Stav projektu.** Panel **Průběh** vypíše detekované kontroly aktuálního projektu ještě předtím, než kdy běžely, tlačítkem **Spustit kontroly projektu** je všechny spustí bez modelu a výsledek uchová u projektu v `.qwen/check-status.json`, takže přežije restart. Každý řádek ukazuje, zda prošel nebo selhal, kdy běžel, a po najetí myší konec výstupu. **Nech agenta opravit selhání** otevře v projektu chat ve Vývoji a zadá agentovi, ať kontroly spustí, opraví co selhává a opakuje to, aniž by testy oslaboval. Obě tlačítka srozumitelně oznámí, když projekt žádné detekovatelné kontroly nemá.

**Automatické commity úloh.** V dialogu projektu (třítečkové menu u výběru projektu) přepínač **Automaticky commitnout každou dokončenou úlohu** způsobí, že každá úspěšně dokončená úloha ve Vývoji nebo Počítači commitne právě své změněné soubory — předmět z cíle úlohy, tělo z finálního souhrnu, závěr s odkazem zpět do chatu. Přepínač je ve výchozím stavu vypnutý a Marvin nikdy nepushuje; v chatu se zobrazí oznámení s hashem commitu.

Dokončená kontrola se zapíše do panelu. Pokud změněná úloha končí bez validace, dokončeného plánu nebo kontroly diffu, harness model jednou upozorní. Model užitečnou kontrolu provede, nebo vysvětlí, proč pro danou úlohu nemá smysl.

Finální kontrola je doporučení, ne omezení. Explicitní zadání uživatele na architekturu, formu, rozsah nebo single-file výsledek má přednost.

# 15. Ovládání počítače

Počítač přidává GUI nástroje ke kompletní sadě Vývoje.

## Cyklus

1. Model zavolá `screenshot`.
2. Obrázek se podle potřeby zmenší a přiloží.
3. Model určí prvky v souřadnicích obrázku.
4. Klikne, pohne myší, scrolluje, píše nebo stiskne klávesy.
5. Dalším screenshotem ověří výsledek.

Harness přepočítá souřadnice obrázku na skutečné rozlišení primárního displeje.

## GUI akce

- Screenshot a informace o obrazovce.
- Pohyb myši pro hover.
- Levé, pravé, prostřední kliknutí a dvojklik.
- Scroll na volitelné pozici.
- Přímé psaní ASCII nebo vložení Unicode/dlouhého textu přes schránku.
- Vypsat otevřená okna a vyfotit jedno z nich podle jména, i když ho zakrývá jiný

program. Tak model sleduje program, který spustil, aniž by vám bral obrazovku: běžný screenshot ukazuje to, co je vpředu, což obvykle není testovaný program.

- Poslat klávesy přímo do jednoho programu podle titulku okna, bez přepnutí do

popředí. Změřeno na zakrytém herním okně: stisky, uvolnění i kombinace jako Ctrl+S dorazí, zatímco okno, ve kterém pracujete, nedostane nic. Tak model ovládá program, který spustil, a vy můžete dělat něco jiného.

- Přepnout okno do popředí podle jména a obnovit ho, pokud bylo minimalizované.

To je záloha pro programy, které čtou klávesnici přímo místo přes okenní zprávy a jde k nim dostat jen ve popředí — bere vám to obrazovku, takže model zkouší nejdřív cestu výše. Když systém přepnutí odmítne, model se to dozví, místo aby hlásil úspěch.

- Spustit program na SDL nebo pygame úplně bez okna, když jde o chování a ne

o vzhled: kreslí do paměti, uloží snímek jako obrázek, na který se model podívá, a vaší obrazovky se nedotkne.

- Klávesy a kombinace jako Enter, Escape, Ctrl+S, Alt+F4 nebo Win+D. Klávesy

odcházejí jako hardwarové scancody, které hry a další okna na SDL nebo DirectInput vyžadují: rozlišují klávesy podle scancodu a klávesu bez něj ignorují, což je důvod, proč takové okno dřív nereagovalo na nic. Klávesa se navíc chvíli drží stisknutá, aby neproletěla mezi dvěma dotazy okna. Když hra pořád nereaguje, lze si vyžádat delší podržení; pro vzácné okno, které to preferuje, zůstává k dispozici i starší způsob. Failsafe levého horního rohu platí pro oba.

### Failsafe

Rychlým přesunutím fyzického kurzoru do levého horního rohu primární obrazovky aktivujete PyAutoGUI failsafe a přerušíte GUI akce.

VAROVÁNÍ: Počítač vidí obrazovku a může provádět destruktivní akce. Pro e-mail, platby, účty, mazání a citlivé workflow používejte Supervised autonomii.

## 16. Autonomie a potvrzování

| Úroveň | Chování |
|---|---|
| Supervised | Každý zápis, shell akce a GUI akce vyžaduje potvrzení; čtení pokračuje. |
| Semi | První WRITE akce v úloze vyžádá potvrzení; potom úloha pokračuje. |
| Auto | WRITE akce pokračují bez potvrzení. |

V produkci není limit agentních kroků.

Při potvrzení chat vypíše čekající akce a zobrazí **Povolit** a **Zamítnout**. Zamítnutí se vrátí modelu, aby mohl zvolit jiný postup.

Přesměrování, instalace balíčků, libovolný Python, kopírování, mazání, síťové zápisy, Git commit/push a smíšené příkazy se konzervativně považují za WRITE.

## 17. Terminálové UI a CLI

Instalátor přidá **Marvin (CLI)** do nabídky Start. Spouští `run_cli.bat` a interaktivní TUI.

| Příkaz | Účel |
|---|---|
| `/memory` | Zobrazit globální, režimovou a projektovou paměť. |
| `/model q4\|q5\|ornith_q5` | Přepnout model a restartovat server. |
| `/work discussion\|research\|writing\|development\|computer` | Zvolit pracovní režim. |
| `/mode chat\|agent\|computer` | Legacy kompatibilní režimová zkratka. |
| `/autonomy supervised\|semi\|auto` | Nastavit potvrzování. |
| `/thinking xhigh\|medium\|low\|off` | Nastavit hloubku uvažování. |
| `/ws [cesta]` | Zobrazit nebo nastavit workspace. |
| `/img <cesta>` | Přiložit obrázek k dalšímu promptu. |
| `/screenshot` | Zachytit obrazovku. |
| `/new`, `/sessions`, `/load <id>` | Spravovat relace. |
| `/server status\|start\|stop` | Ovládat inference server. |
| `/help`, `/exit` | Nápověda a ukončení. |

Při potvrzení y povolí, n zamítne a a povolí zbývající WRITE akce v úloze. Ctrl+C přeruší generování.

# 18. Úložiště, zálohy a logy

| Cesta | Obsah |
|---|---|
| `runtime\models` | GGUF modely a vision projektory |
| `runtime\llama` | CUDA binárky `llama.cpp` |
| `runtime\workspace-settings.json` | Aktuální nastavení webového pracovního okna a poslední úspěšný model |
| `runtime\application.sqlite3` | Fronta úloh, trvalé události a registr souborů |
| `runtime\models\strata` | Váhy Flash-Next a soubory z nich připravené |
| `runtime\strata` | Jádro Flash-Next, připravené automaticky |
| `sessions\<id>` | Zprávy, přílohy, task state, research, komprese a exporty |
| `projects` | Projekty vytvořené aplikací |
| `projects.json` | Registr projektů |
| `memory` | Globální a režimová paměť |
| `<projekt>\QWEN_MEMORY.md` | Projektová paměť |
| `skills`, `user-skills`, `<projekt>\.qwen-skills` | Tři vrstvy skillů |

Zálohujte projektové adresáře, `sessions`, `memory`, `user-skills`, `projects.json` a stavové soubory v `runtime`. Starší `webui-state.json` slouží pro kompatibilitu a převzetí dřívějších voleb; hlavní UI používá `workspace-settings.json`.

## Kompletní offline záloha instalace

Aplikace umí vytvořit přenosnou instalační zálohu přímo ze souborů, které už jsou na tomto počítači. Otevřete **Nastavení > Data a zálohy**, zvolte **Vytvořit zálohu** a vyberte nadřazený adresář, ideálně na jiném disku. Vznikne časově označená složka `QwenHarness-Offline-Backup-*`, která obsahuje:

- Všechny kompletní modely a vision projektory z `runtime\models`.
- Nainstalované `llama.cpp` a CUDA runtime z `runtime\llama`.
- Aktuální výběr modelů a `requirements.txt`.
- Kopii Python balíčků, které už jsou nainstalované v `.venv`.
- Odpovídající Setup.exe, pokud je dostupný v aktuálním build adresáři; verze 1.8 přednostně přikládá Full s vlastním Pythonem.
- `manifest.json` s velikostí a SHA-256 hashem každého zálohovaného souboru.

Modely, runtime i Python závislosti se kopírují přímo z aktuálního adresáře Marvin. Při tvorbě zálohy se nic z toho znovu nestahuje. Je potřeba přibližně tolik volného místa, kolik zabírá nainstalovaný runtime; operace může trvat, protože se každý soubor kontrolně hashujete.

## Instalace z offline zálohy

1. Zkopírujte celou offline složku, včetně `manifest.json`, `payload`, `python-dependencies` a přiloženého instalátoru. Kompletní balíček se všemi současnými modely potřebuje přes 200 GB místa.
2. Spusťte přiložený **Full** instalátor. Obsahuje vlastní Python a nevyžaduje jeho samostatnou instalaci. Jen pokud používáte **Minimal**, musíte mít 64bitový Python 3.12 připravený předem.
3. Použijte jednu z možností:

- Pokud je Setup.exe uvnitř zálohy vedle `manifest.json`, spusťte ho přímo tam; zálohu rozpozná automaticky.
- Položte zálohu vedle Setup.exe a přejmenujte ji přesně na `QwenHarness-Offline-Backup`; instalátor ji rozpozná automaticky.

- Nainstalujte Marvin, v nabídce Start spusťte **Instalace z offline zálohy** a vyberte složku zálohy.

4. Když instalátor rozpozná zálohu vedle sebe, první příprava použije místní obnovu přednostně. Úplná obnova přenese všechny modely obsažené v balíčku, včetně Flash-Next a jeho jádra, a kontroluje jejich SHA-256. Výběr modelů v průvodci řídí případné další stahování.
5. Příkaz **Instalace z offline zálohy** v nabídce Start také obnovuje místní data přednostně. Běžné nastavení bez této volby používá internet a zaregistrovanou zálohu jako náhradní zdroj. Pokud součást není ani místně, dokončení vyžaduje její získání.

Tlačítkem **Použít jako zálohu** zaregistrujete složku pro případ budoucího selhání downloadu, **Ověřit SHA-256** zkontroluje každý soubor podle manifestu a **Zapomenout výběr** tuto pojistku vypne. Výběr zálohy nezakazuje přístup k internetu. Setup.exe, `manifest.json`, `README-OFFLINE.txt`, `requirements.txt`, `python-dependencies` a `payload` musí zůstat společně v jedné záložní složce.

Offline instalační záloha neobsahuje chaty, projekty, paměť ani osobní skills. Tato nenahraditelná uživatelská data zálohujte samostatně podle seznamu výše.

| Log | Použití |
|---|---|
| `runtime\launcher.log` | Launcher a start aplikace |
| `runtime\app.log` | Nativní aplikace a crashe |
| `runtime\webapp.log` | Web UI a Python chyby |
| `runtime\llama-server.log` | Model, kontext, CUDA, inference a timing |
| `runtime\strata-server.log` | Načítání a inference Flash-Next |

# 19. Řešení problémů

## Server nestartuje

Otevřete **Nastavení > Model a zařízení**, přečtěte hlášení a zkuste restart. Ověřte kompletní model, kompatibilní NVIDIA ovladač a volnou paměť. Flash-Next drží váhy v systémové RAM; další velké otevřené programy zmenšují, co může použít, a potřebuje ovladač NVIDIA 580 nebo novější. Podrobnosti jsou v `runtime\llama-server.log`, u Flash-Next v `runtime\strata-server.log`.

## Chybí model nebo projektor

Spusťte **Instalace prostředí a modelů** z nabídky Start. Hotové soubory se přeskočí a podporovaný nedokončený download se obnoví.

## Port Web UI je obsazený

Launcher najde blízký volný port nebo znovu použije existující Marvin Web UI. URL je v `launcher.log`.

## Dlouhý chat je pomalý nebo přetekl

Zkontrolujte kontext, použijte Q5/Q8 192k nebo Q4/Q8 256k, ručně komprimujte, předejte do nového chatu nebo odstraňte nepotřebné pins/obrázky. Agent při overflow jednou automaticky komprimuje a zopakuje request.

## UI vypadá zamrzle

Sledujte fázi a uplynulý čas: může probíhat stahování, kontrola vah, zpracování dlouhého vstupu, thinking nebo nástroj. U Flash-Next může nový velký vstup trvat od desítek sekund do několika minut podle RAM a karty. **Zastavit úlohu** ukončí aktuální požadavek; samostatný proces na pozadí zastavte v **Průběh > Procesy**. Modelový server se zastavuje zvlášť v **Model a zařízení**.

## Izolovaný browser nestartuje

- Microsoft Edge musí být nainstalován ve standardním umístění Windows.

- Po upgradu spusťte opravu/aktualizaci prostředí, aby se nainstaloval Python Playwright.
- Session je headless a nepoužívá běžné Edge/Chrome okno uživatele.

### Obnovení po restartu

V **Rozpracovaná úloha** zvolte **Pokračovat v úloze**. Podle možnosti se obnoví pending potvrzení, částečný text, journal a procesní metadata.

### Kde je PDF/DOCX

S projektem v `<projekt>\exports`, bez projektu v `sessions\<id>\exports`. Exportovaný soubor najdete také v pravé záložce **Výsledky**.

### Jazyk se nezměnil celý

Použijte **Nastavení > Vzhled a jazyk**. UI se přepne přímo a zachová chat. Nativní launcher může vyžadovat restart. Uložená UI volba má přednost před jazykem instalátoru.

## 20. Reference nástrojů

Nástroje běžně požadujete přirozeným jazykem.

### Všechny režimy

| Nástroj | Schopnost |
|---|---|
| `read_memory`, `save_memory` | Číst a ukládat tři vrstvy paměti. |
| `web_search`, `web_fetch` | Hledat a číst veřejný web/dokumenty. |
| `context_status` | Tokeny, zprávy, obrázky, pins a komprese. |
| `pin_context_file`, `unpin_context_file` | Spravovat pins aktuálního chatu. |
| `list_project_documents`, `read_project_document` | Katalog a čtení dokumentů. |
| `list_skills`, `read_skill` | Katalog a načtení skillů. |
| `export_document` | Export Markdown, DOCX nebo PDF. |
| `list_dir`, `read_file` | Procházet adresáře a číst rozsahy textu. |
| `search_files`, `find_files` | Rychlé literal/regex hledání a glob souborů. |
| `write_file`, `apply_patch` | Vytvářet a atomicky upravovat text. |
| `make_directory`, `move_file`, `delete_file` | Strukturované změny s rollbackem. |
| `list_task_changes`, `undo_task_changes` | Task journal a rollback. |
| `view_image` | Vizuální analýza lokálního obrázku. |

### Sledování dokumentu, který se právě píše

V seznamu **Změněné soubory** má dokument, kterého se úloha dotkla, druhé tlačítko vedle zobrazení rozdílů. Otevře dokument vysázený a **drží ho aktuální, zatímco model píše**: náhled soubor znovu načítá a překresluje jen tehdy, když se obsah opravdu změnil, takže neproblikává.

Tlačítko se objeví u toho, co jde vysázet — Markdown, text, DOCX, PDF, HTML, CSV a tabulky. Zdrojový kód a binární soubory nemají co ukázat, takže u nich zůstává jen zobrazení rozdílů.

To je odpověď na jedinou otázku, kterou režim Psaní během práce neumělo zodpovědět: jak to teď vypadá.

## Psaní, Vývoj a Počítač

| Nástroj | Schopnost |
|---|---|
| `task_plan_status`, `set_task_plan` | Číst nebo vytvořit operační plán. |
| `update_task_step` | Aktualizovat stav kroků. |
| `record_task_validation` | Zapsat ručně provedenou kontrolu. |

## Vývoj a Počítač

| Nástroj | Schopnost |
|---|---|
| `repo_overview`, `project_instructions` | Mapa repozitáře a hierarchické instrukce. |
| `project_validation_profile`, `start_project_check` | Nabídka a spuštění projektových kontrol. |
| `find_symbol`, `document_symbols`, `find_references` | Deklarace, symboly dokumentu a rychlá použití. |
| `browser_open`, `browser_snapshot` | Otevřít a sémanticky prohlédnout izolovaný Edge. |
| `browser_fill`, `browser_click`, `browser_press`, `browser_wait` | Práce s refs a čekání na změny UI. |
| `browser_select`, `browser_hover`, `browser_scroll` | Dropdowny, hover a scrolling. |
| `browser_upload`, `browser_download` | Lokální upload a uložení downloadu. |
| `browser_viewport` | Desktopový, tabletový nebo mobilní viewport. |
| `browser_console`, `browser_network` | Konzole a síťová diagnostika. |
| `browser_screenshot`, `browser_close` | Vision screenshot a zavření browseru. |
| `git_status`, `git_diff`, `git_commit` | Lokální Git operace bez automatického push. |
| `run_command` | Krátký Bash, PowerShell nebo cmd příkaz. |
| `start_command`, `poll_command`, `send_stdin`, `terminate_command` | Persistentní procesy na pozadí. |

## Počítač

| Nástroj | Schopnost |
|---|---|
| `screenshot`, `get_screen_info` | Obrazovka a souřadnice. |
| `move_mouse`, `click`, `scroll` | Myš a scrolling. |
| `type_text`, `press_key` | Text, klávesy a kombinace. |

# 21. Praktické příklady

```
Diskuze: Porovnej tyto dva produkční postupy bez coding procedur.

Výzkum: Prozkoumej současný stav virtual production stages. Zachovej rozpory,
uveď nejistoty a vyexportuj finální syntézu jako PDF.

Psaní: Přečti treatment v projektu, přepiš druhý akt ve stejném hlasu a ulož DOCX.

Vývoj: Projdi repozitář, reprodukuj chybu, oprav příčinu, spusť testy a shrň změny.

Paměť: Zapamatuj si pro tento projekt, že rendery patří do D:\Show\Delivery.

Připnutí: Připni docs\ARCHITECTURE.md pro tento chat.

Počítač: Udělej screenshot, změň požadované nastavení a ověř výsledek dalším screenshotem.

Export: Ulož existující odpověď jako PDF. Neprováděj nový výzkum.
```

# 22. Provozní omezení

## Trvalé hranice produktu

Následující funkce jsou **výslovně mimo tento produkt a nikdy nebudou přidány**, pokud vlastník osobně nezmění toto produktové rozhodnutí:

- Language servery nebo distribuční/runtime vrstva LSP. Určeným řešením je vestavěný

lehký symbolový index.

- Persistentní interaktivní terminál jako hlavní workflow. Shell zůstává záložní nástroj.
- Paralelní modeloví agenti, subagenti nebo multi-model orchestrace. GPU používá jeden

lokální model a pracuje sekvenčně.

- Kontext s jedním milionem tokenů. Kontext zůstává v praktických profilech lokální GPU.
- Obecný plugin host, MCP ekosystém nebo široký integrační framework.

Tyto body do Marvin nepatří. Nejsou odloženým roadmap backlogem.

- Jeden velký model zabere téměř celou GPU, proto je inference jednoslotová.
- Čtecí nástroje a procesy na pozadí mohou běžet souběžně.
- Přepnutí modelu/KV restartuje server a dočasnou prompt cache, nikoli historii.
- Kontextová čísla jsou odhady; tokenizace a cena obrázků závisejí na modelu.
- Model se může mýlit; ověřovací nástroje nezaručují správnost.
- Computer mode pracuje s primárním monitorem a viditelným UI.
- Web může selhat na login walls, JavaScript-only webech, CAPTCHA nebo blokovaných downloadech.
- Veřejné skills před instalací zkontrolujte, protože ovlivňují postup modelu.
- Git commit zůstává lokální bez výslovného požadavku na push.
- Pravidelně zálohujte nenahraditelné projekty a relace.

# 23. Rozšířené pracovní operace

## Čtení celých dokumentů

Modelu můžete zadat konkrétní stránku PDF, blok DOCX, list a oblast Excelu nebo rozsah řádků CSV. Čtecí nástroj vrací také údaj pro navazující část, takže se práce nemusí zastavit na prvních sto řádcích. Excel umí zobrazit uložené hodnoty nebo vzorce; čtecí nástroj sám nepřepočítává vzorce.

Skenované stránky PDF a diagramy lze přes `view_document_page` vykreslit a prohlédnout aktivním modelem s vision. Existující Word soubor upravuje `edit_word_document` se zachováním odstavců, tabulek a stylů textových úseků.

## Dohledání původního kontextu

Komprese odpovídá režimu Diskuze, Výzkum, Psaní, Vývoj nebo Počítač. Zpracují se všechny části vstupu a mezisouhrny se ukládají. Původní zprávy lze dohledat přes `search_chat_history` a `read_chat_history`, i když už nejsou v aktivním kontextu.

Výzkum ukládá průběžné poznámky k důkazům. Po přerušení je syntéza může znovu použít. Citace otevře text původního zdroje; nalezené, ale nenačtené položky zůstávají viditelné bez filtru důvěryhodnosti.

## Body obnovy

Otevřete **Výsledky > Body obnovy** nebo použijte `/checkpoint název`. Bod zachytí pracovní soubory, nikoli generované závislosti nebo modely a runtime. Úlohový snapshot eviduje také změny provedené příkazy modelu. **Obnovit** oznámí konflikt s pozdější úpravou a nepřepíše ji tiše. Částečné selhání se neoznačí za kompletní obnovení.

Tlačítko porovnání u bodu obnovy vypíše všechno, co se v projektu od jeho pořízení změnilo — upravené, smazané i nově vzniklé soubory — a každý řádek otevře prohlížeč rozdílů proti uložené verzi. Nevyžaduje to žádnou činnost modelu, takže bod obnovy slouží i k revizi vlastních úprav. Soubor vzniklý až po bodu obnovy za sebou uloženou verzi nemá: lze ho zobrazit, ale ne vrátit.

## Přenos projektu

Použijte **Nastavení > Data a zálohy > Exportovat projekt**. ZIP zahrnuje projektové soubory, projektovou paměť a skilly, historii chatů, přílohy a odkazy na rozhodnutí. **Importovat projekt** vytvoří nový adresář a nové identifikátory chatů a upraví uložené cesty příloh i odkazy na původní konverzace.

Projektový archiv je oddělený od zálohy modelů a runtime. Ta nadále funguje jako pojistka po selhání internetového zdroje; lze ji zvolit i přímo pro offline instalaci.

## Vývoj ze zdrojů a vydání

Běžný uživatel volí Minimal nebo Full; Full obsahuje Python 3.12 i závislosti. Node.js není nutný pro provoz aplikace. Vývojář sestaví frontend příkazy `npm --prefix frontend ci` a `npm --prefix frontend run build`. Python obsluhuje výsledný adresář `ui_dist`. Ověřené Windows verze balíčků drží `requirements-windows-py312.lock`.

Pracovní plocha je jediné rozhraní. Původní Gradio bylo ve verzi 1.16.0 odstraněno i se závislostí, kterou potřebovalo.